

Paimon-Iceberg

模块核心架构设计文档

核心架构设计 · 设计模式 · 流程图 · 时序图 · 调用链

模块：paimon-iceberg

分支：release-1.3.1

版本：Apache Paimon 1.5-SNAPSHOT / Iceberg 1.8.1

生成工具：AI Architecture Analyzer

生成日期：2026年03月27日

第1章 模块概述

1.1 模块定位

paimon-iceberg 是 Apache Paimon 数据湖存储引擎中的一个轻量级集成模块，核心职责是将 Paimon 表的元数据通过 Iceberg REST Catalog API 提交到外部 Iceberg REST 目录服务中，从而使 Iceberg 生态的读取引擎（如 Spark、Trino、Presto 等）能够直接访问 Paimon 的原始数据文件，实现 Paimon 与 Iceberg 生态的元数据互通。

该模块是 Paimon 多 Catalog 兼容性战略的关键组成部分。Paimon 目前支持四种 Iceberg 元数据存储方式：TABLE_LOCATION（本地表目录）、HADOOP_CATALOG、HIVE_CATALOG（Hive Metastore）和 REST_CATALOG（REST Catalog），paimon-iceberg 模块专门负责 REST_CATALOG 路径的实现。

1.2 核心目标

- 元数据桥接：将 Paimon 快照元数据转换并提交到 Iceberg REST Catalog，实现跨引擎数据读取
- Schema 兼容：解决 Paimon（fieldId 从0开始）和 Iceberg（fieldId 从1开始）之间的 Schema ID 偏移问题
- 增量同步：支持基于 Base Metadata 的增量元数据更新，避免全量重建
- 容错恢复：当检测到 Base Metadata 不一致时，自动重建 Iceberg 表以恢复同步
- SPI 可插拔：通过 Java SPI 机制实现工厂模式，可动态发现并加载 REST Committer

1.3 源码文件一览

文件名	类型	大小	职责描述
IcebergRestMetadataCommitter.java	核心实现	20.36 KB	REST Catalog 元数据提交器：Schema 适配、表创建/更新、快照同步
IcebergRESTMetadataCommitterFactory.java	SPI 工厂	1.23 KB	SPI 工厂类，创建 IcebergRestMetadataCommitter 实例
META-INF/services/...Factory	SPI 配置	844 B	注册工厂到 Java SPI 服务发现机制

1.4 测试文件一览

文件名	大小	类型	说明
IcebergRestMetadataCommitterTest	36.61 KB	单元测试	分区/非分区、Schema变更、快照过期、Base不一致等
IcebergMetadataTest	27.72 KB	单元测试	元数据读写兼容性：复杂Schema、分区、排序

文件名	大小	类型	说明
IcebergDVCompatibilityTest	12.88 KB	兼容性测试	Deletion Vector 兼容性，Iceberg V3 格式支持
FlinkIcebergITCase	5.17 KB	集成测试	Flink + Hadoop Catalog DV 端到端验证
IcebergRestMetadataCommitterITCase	5.51 KB	集成测试	Flink + REST Catalog 端到端验证

第2章 核心架构设计

2.1 整体架构图

paimon-iceberg 模块采用分层架构设计，从上到下依次为：Paimon 核心层调用 CommitCallback 回调接口，通过 SPI 工厂机制发现并创建 IcebergMetadataCommitter 实现，最终通过 Iceberg REST Catalog SDK 与外部 REST 服务交互。

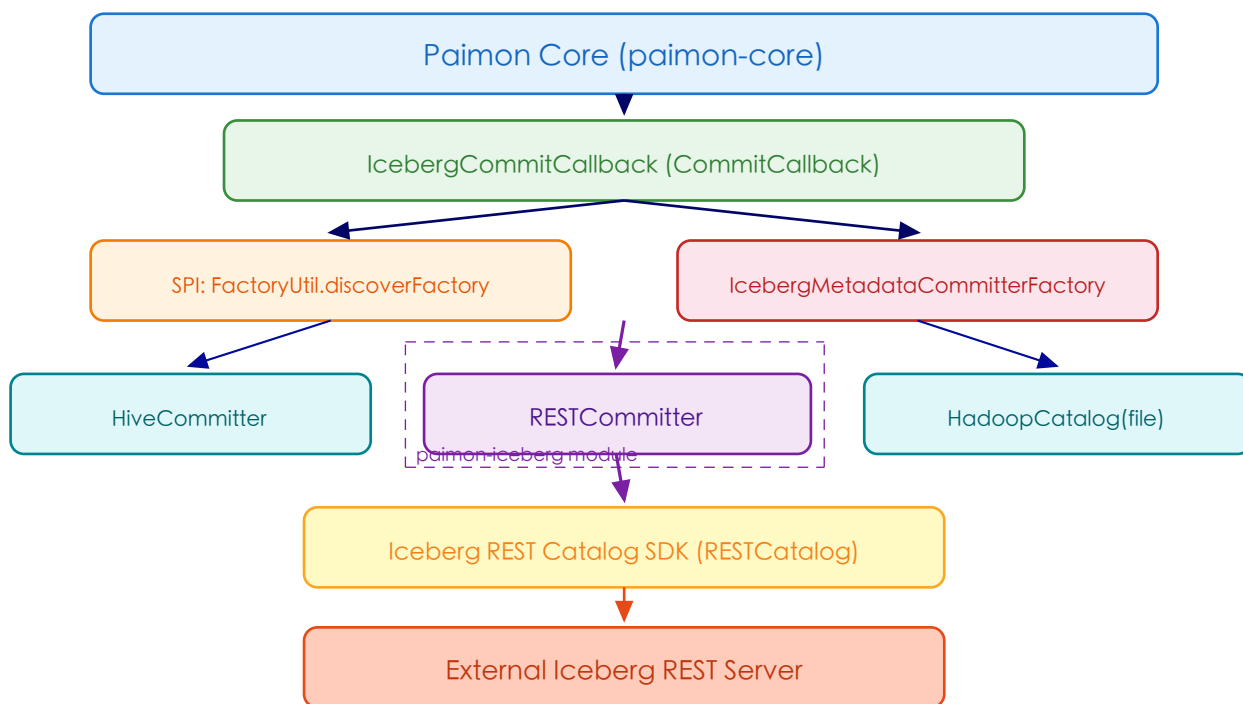


图2-1 paimon-iceberg 模块整体分层架构图

2.2 类层次结构

模块的类继承与实现关系如下：

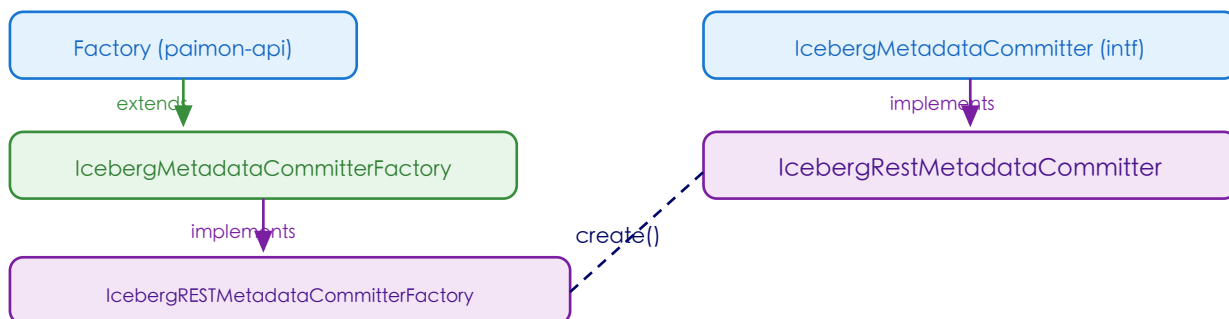


图2-2 类继承与实现关系图

2.3 Paimon 与 Iceberg 概念映射

Paimon 概念	Iceberg 概念	映射说明
Snapshot	Snapshot	每个 Paimon 快照对应一个 Iceberg 快照
IcebergMetadata	TableMetadata	通过 toJson() 进行 JSON 序列化转换
IcebergSchema (id从0起)	Schema (id从1起)	REST Committer 自动 +1 偏移
FileStoreTable	Table (BaseTable)	Paimon 表对象，承载配置和数据
Identifier (db.table)	TableIdentifier (ns.table)	通过 Namespace 进行转换
CommitCallback	TableOperations.commit()	Paimon 提交回调触发 Iceberg 元数据同步
Deletion Vector (V3)	Puffin DV (V3)	仅在 format-version=3 且 bitmap64=true 时同步

第3章 核心类设计模式详解

3.1 IcebergRESTMetadataCommitterFactory — 工厂方法模式 (Factory Method)

模式说明：该类实现了 `IcebergMetadataCommitterFactory` 接口（继承自 `Paimon` 的 `Factory`），通过 `Java SPI (ServiceLoader)` 在运行时动态发现。`IcebergCommitCallback` 初始化时通过 `FactoryUtil.discoverFactory()` 根据 `StorageType` 配置值自动加载对应工厂。

设计意图：将 `Committer` 的创建逻辑与使用逻辑分离，支持 `REST/Hive/Hadoop` 可插拔切换，新增类型仅需添加 `Factory` 实现和 `SPI` 注册。

- `identifier()` — 返回 'rest-catalog'，作为 `SPI` 唯一标识符
- `create(FileStoreTable)` — 根据 `Paimon` 表创建 `IcebergRestMetadataCommitter`

3.2 IcebergRestMetadataCommitter — 策略模式 + 适配器模式

策略模式：作为 `IcebergMetadataCommitter` 接口的 `REST` 策略实现，与 `IcebergHiveMetadataCommitter` (`Hive` 策略) 形成策略族，由 `IcebergCommitCallback` 根据配置选择。

适配器模式：`adjustMetadataForRest()` 将 `Paimon` 的 `IcebergMetadata` (`Schema ID` 从0起) 适配为 `Iceberg TableMetadata` (`Schema ID` 从1起)，解决两者的 `field ID` 偏移不兼容问题。

- `commitMetadataImpl()` — 核心逻辑：适配元数据 → 判断表存在 → 创建/增量更新
- `adjustMetadataForRest()` — 适配器核心，`Schema ID` 和 `Snapshot Schema ID` 全部 +1
- `updatesForCorrectBase()` — 正常基线增量更新策略
- `updatesForIncorrectBase()` — 基线不一致时全量重建策略

3.3 commitMetadataImpl — 模板方法模式 (Template Method)

该方法定义了固定的算法骨架：1) `adjustMetadataForRest` 适配 → 2) 检查/创建数据库 → 3) 判断表是否存在 → 4) 根据基线正确性选择更新策略 → 5) 构建并提交更新到 `REST Server`。

3.4 createTable — 策略分支模式

根据分区 `fieldId` 位置采取不同建表策略：

- 无分区或 `fieldId=0`：创建空 `Schema` 表，通过 `Schema` 演化更新
- `fieldId>0`：创建 `Dummy Schema + PartitionSpec` 的表，预填充占位字段避免 `partition evolution`

3.5 SPI 服务注册 — 服务定位器模式

通过

META-INF/services/org.apache.paimon.factories.Factory

注册工厂类，FactoryUtil.discoverFactory() 在运行时通过 ServiceLoader 扫描 classpath，根据 identifier() 匹配 StorageType，实现零耦合服务发现。

第4章 核心流程图

4.1 元数据提交主流程 (commitMetadataImpl)

以下是核心提交方法的完整执行流程：

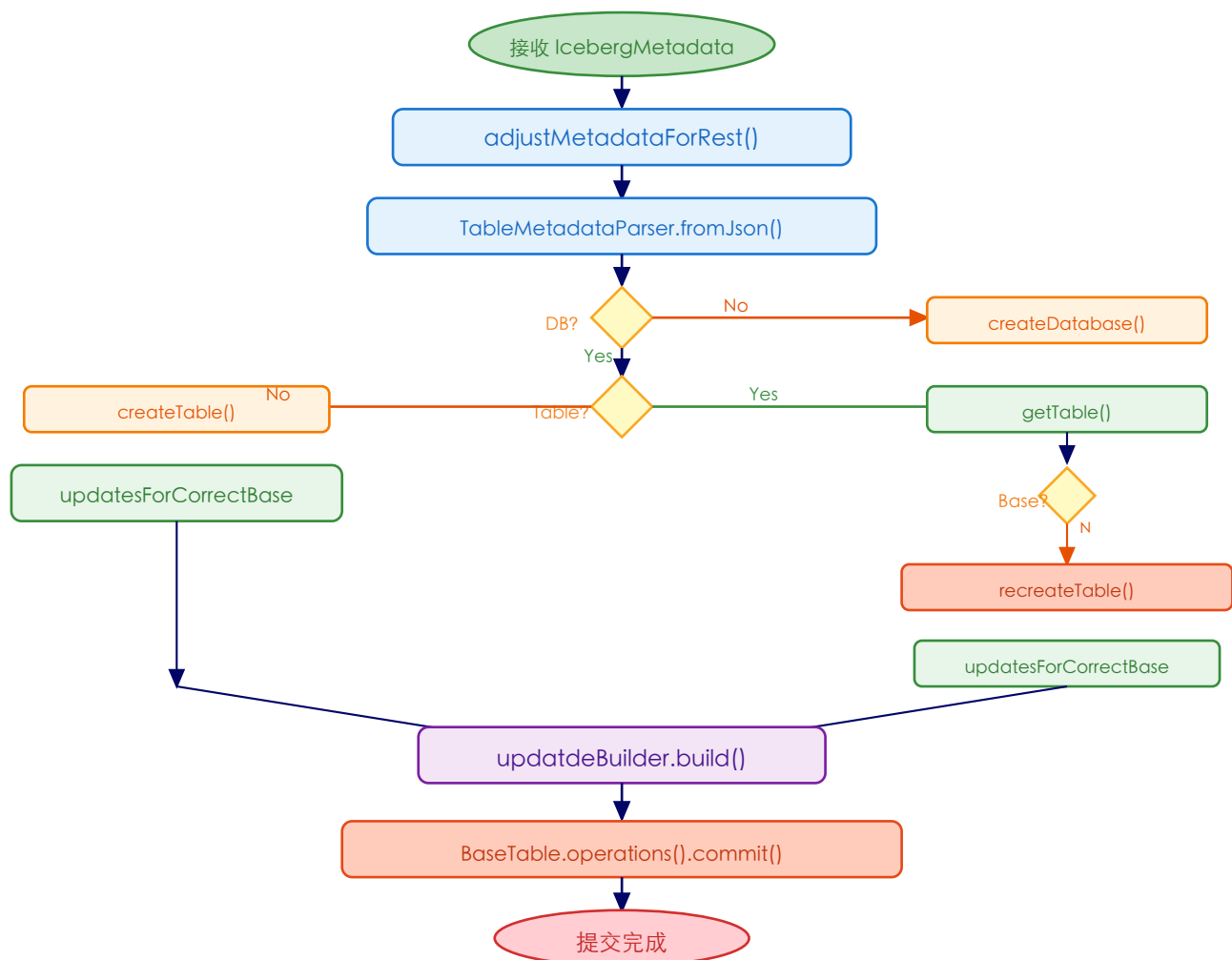


图4-1 元数据提交主流程图

4.2 表创建策略流程 (createTable)

createTable() 根据分区字段 fieldId 位置采取不同建表策略：

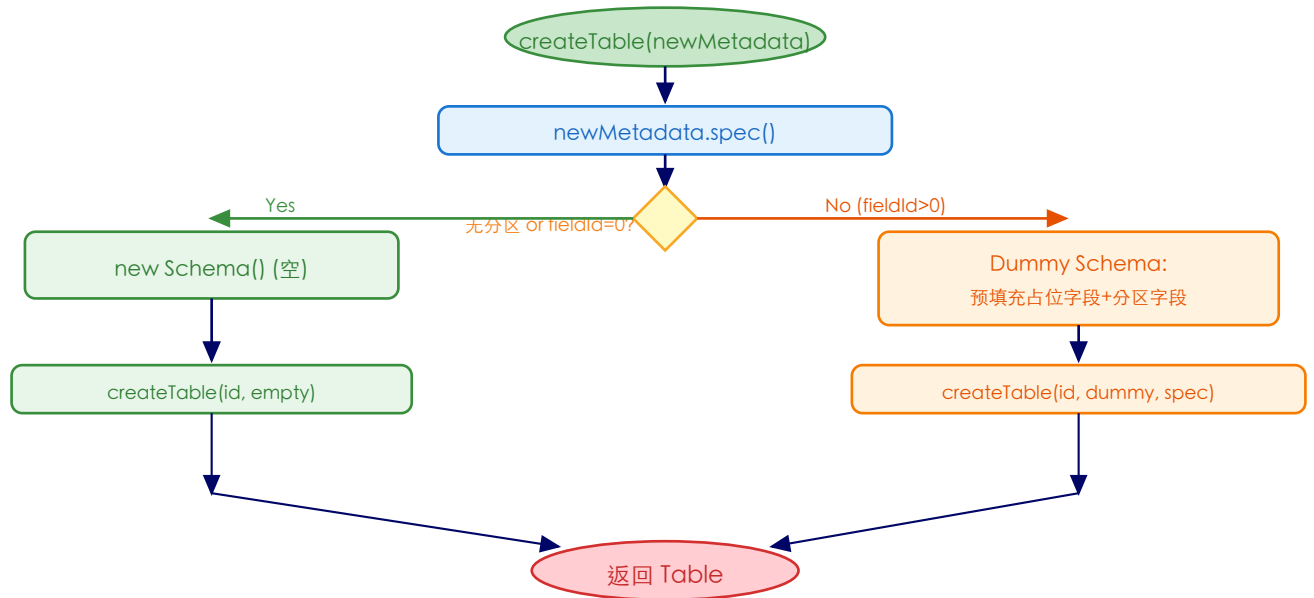


图4-2 表创建策略流程图

第5章 核心时序图

5.1 首次元数据提交时序图（新表创建）

Paimon 表首次提交快照时，创建 Iceberg 表并同步元数据的完整交互序列：

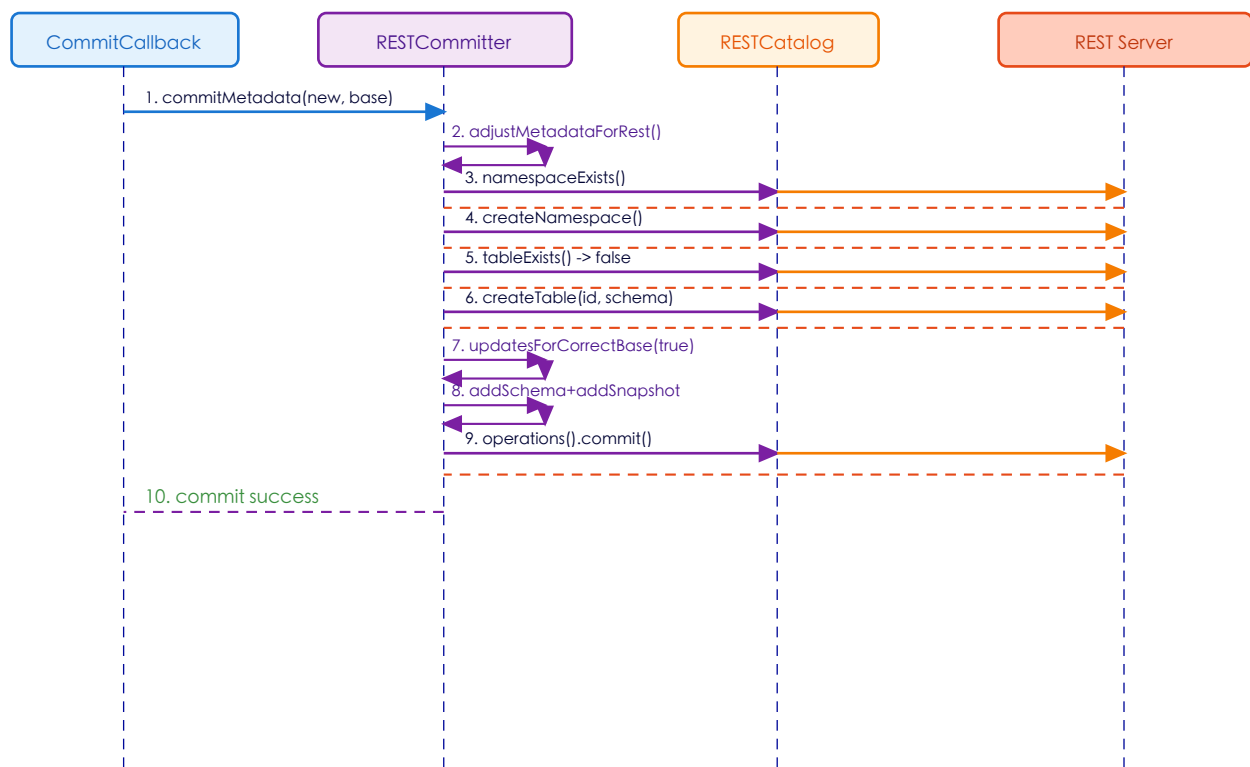


图5-1 首次元数据提交时序图（新表创建场景）

5.2 增量元数据更新时序图（Base 正确）

表已存在且 Base 正确时（快照 ID 连续），执行增量更新：

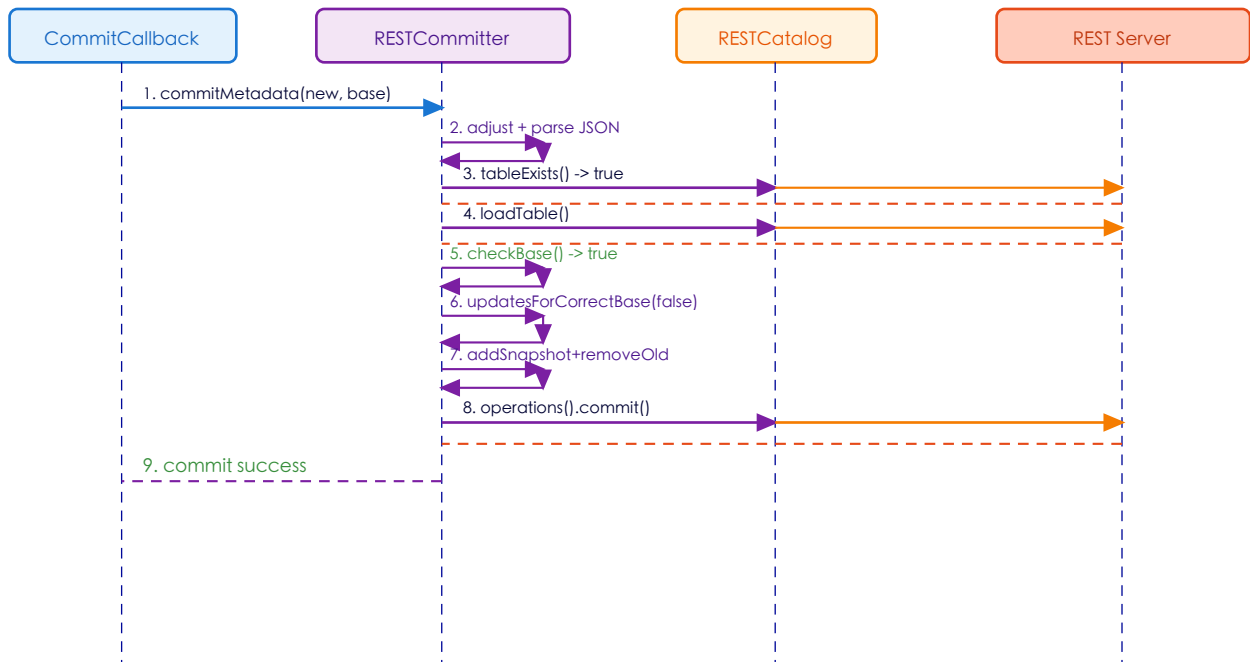


图5-2 增量元数据更新时序图（Base 正确场景）

5.3 Base 不一致恢复时序图

Base 不一致时，系统先删除再重建 Iceberg 表：

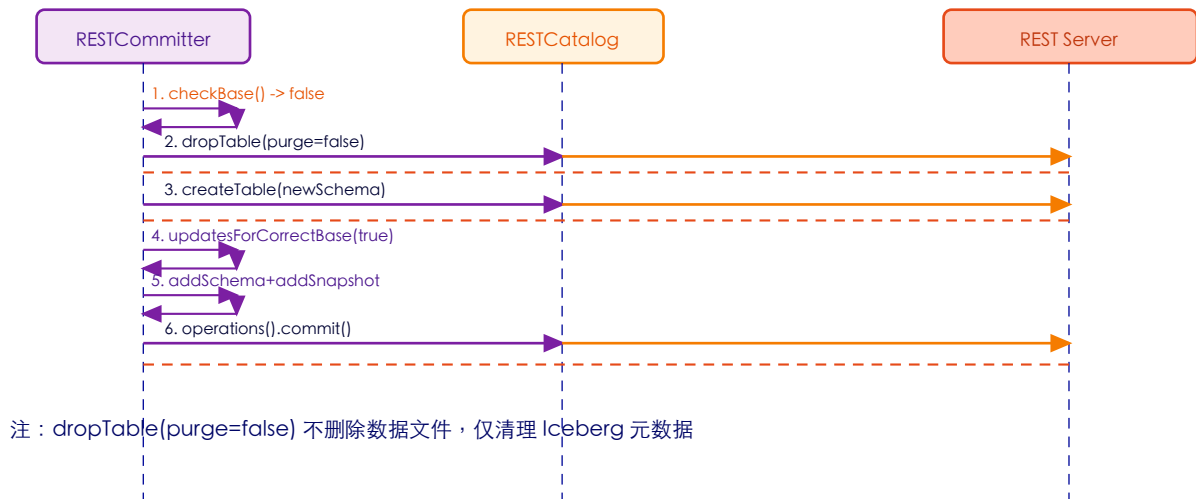


图5-3 Base 不一致恢复时序图

第6章 关键场景调用链

6.1 场景一：首次快照提交并同步到 REST Catalog

触发条件：Paimon 表配置 metadata.iceberg.storage=rest-catalog，首次写入数据并提交。

步骤	类.方法	说明
1	TableCommitImpl.commit()	Paimon 提交快照，触发 CommitCallback
2	IcebergCommitCallback.call(Context)	回调入口，创建 Iceberg 元数据
3	FactoryUtil.discoverFactory()	SPI 加载 IcebergRESTMetadataCommitterFactory
4	IcebergRESTMetadataCommitterFactory.create()	创建 IcebergRestMetadataCommitter
5	IcebergRestMetadataCommitter.commitMetadata()	调用提交接口
6	adjustMetadataForRest()	Schema ID +1 偏移适配
7	createTable(newMetadata)	根据分区策略创建 Iceberg 表
8	updatesForCorrectBase(isNew=true)	构建全量元数据更新
9	BaseTable.operations().commit()	提交到 REST Server

6.2 场景二：增量快照提交（Base 正确）

触发条件：表已同步过元数据，后续快照 snapshotId 连续。

步骤	类.方法	说明
1	commitMetadataImpl()	入口方法
2	adjustMetadataForRest()	Schema ID 偏移适配
3	loadTable()	加载现有 Iceberg 表
4	checkBase() -> true	快照 ID 连续，基线正确
5	updatesForCorrectBase(false)	增量更新：addSnapshot + removeOld
6	BaseTable.operations().commit()	提交增量更新

6.3 场景三：Base 不一致恢复

触发条件：中途禁用 Iceberg 兼容后重新启用，快照 ID 不连续。

步骤	类.方法	说明
1	checkBase() -> false	快照 ID 不连续
2	updatesForIncorrectBase()	触发重建策略
3	dropTable(purge=false)	删除 Iceberg 元数据（保留数据文件）
4	createTable(newMetadata)	重新创建 Iceberg 表
5	updatesForCorrectBase(isNew=true)	全量构建更新
6	BaseTable.operations().commit()	提交到 REST Server

6.4 场景四：Schema 变更同步

触发条件：Paimon 表发生 Schema 变更（如添加列）。

步骤	类.方法	说明
1	SchemaManager.commitChanges(addColumn)	Paimon Schema 变更
2	adjustMetadataForRest()	新 Schema ID 也加 1
3	检测 currentSchemaId 变化	发现 Schema 版本升级
4	addAndSetCurrentSchema()	添加新 Schema 并设为当前
5	update.setProperties()	同步 metadata 配置属性
6	addNewSnapshot() + commit()	提交 Schema + 快照更新

6.5 场景五：Flink 端到端 REST Catalog 验证

触发条件：Flink SQL 创建 Paimon 表并配置 rest-catalog。

步骤	操作	说明
1	CREATE TABLE ... WITH ('metadata.iceberg.storage'='rest-catalog')	创建 Paimon 表
2	INSERT INTO paimon.default.T VALUES ...	写入并触发 Iceberg 同步
3	CREATE CATALOG iceberg WITH ('type'='iceberg','catalog-type'='rest')	创建 Iceberg Catalog
4	SELECT * FROM iceberg.default.T	通过 Iceberg 读取，验证数据一致

第7章 配置与扩展

7.1 核心配置属性

配置项	默认值	说明
metadata.iceberg.storage	disabled	存储方式：disabled/table-location/hadoop-catalog/hive-catalog/rest-catalog
metadata.iceberg.format-version	2	Iceberg 格式版本（2或3），V3 支持 DV
metadata.iceberg.rest.*	(无)	REST Catalog 配置前缀
metadata.iceberg.database	(无)	Iceberg 数据库别名
metadata.iceberg.table	(无)	Iceberg 表别名
metadata.iceberg.delete-after-commit.enabled	true	提交后删除旧元数据
metadata.iceberg.previous-versions-max	0	保留历史版本数（REST 至少1）
metadata.iceberg.manifest-compression	snappy	Manifest 压缩算法

7.2 REST Catalog 配置示例

```
-- Flink SQL
CREATE TABLE paimon.`default`.T (
  pt INT, k INT, v INT,
  PRIMARY KEY (pt, k) NOT ENFORCED
) PARTITIONED BY (pt) WITH (
  'metadata.iceberg.storage' = 'rest-catalog',
  'metadata.iceberg.rest.uri' = 'http://iceberg-rest:8181',
  'metadata.iceberg.rest.warehouse' = 's3://bucket/warehouse'
);
```

7.3 扩展点

- 新增 Catalog 类型：实现 Factory + Committer 接口，注册 META-INF/services 即可
- 数据库/表名映射：通过 metadata.iceberg.database/table 配置跨 Catalog 别名
- REST 配置透传：metadata.iceberg.rest.* 前缀自动去除后传递给 RESTCatalog

第8章 设计亮点与总结

8.1 设计亮点

- Schema ID 偏移适配：adjustMetadataForRest() 优雅解决 Paimon(从0起) 与 Iceberg(从1起) 的 field ID 不兼容
- 智能分区建表策略：createTable() 根据 fieldId 位置采取不同策略，避免 partition evolution 兼容性问题
- Base 一致性检测与自动恢复：checkBase() + updatesForIncorrectBase() 实现元数据同步中断后的自动恢复
- SPI 可插拔架构：REST/Hive/Hadoop 三种 Catalog 完全解耦，新增类型零修改现有代码
- 增量同步设计：正常情况仅同步增量 Schema 和 Snapshot，避免全量重建
- 测试覆盖全面：测试代码(88KB)远超生产代码(22KB)，覆盖分区/Schema变更/快照过期/Base 不一致/DV兼容等场景

8.2 设计限制

- 不支持 Path 方式提交：commitMetadata(Path, Path) 抛 UnsupportedOperationException，REST 模式必须使用对象方式
- Schema ID 偏移硬编码：+1 偏移逻辑写死在代码中，如果 Iceberg REST 创建表行为变化需要调整
- Base 不一致时全量重建：recreateTable 会 DROP + CREATE，如果 REST Server 有权限限制可能失败

8.3 质量评估

评估维度	评分	说明
代码结构	★★★★☆	分层清晰，核心仅2个类，职责单一明确
设计模式运用	★★★★★	工厂方法+策略+适配器+模板方法，模式运用恰当且不过度设计
可扩展性	★★★★★	SPI 可插拔，新增 Catalog 类型零修改
容错能力	★★★★☆	Base 不一致自动恢复，但重建策略较激进
测试覆盖度	★★★★★	测试代码4倍于生产代码，场景全面
文档完整度	★★★★☆	类注释简洁，缺少详细 Javadoc